Scientific Machine Learning

Alexander Heinlein[1]

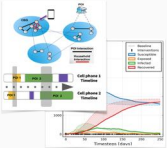
Benchmarking for Scientific Machine Learning in Subsurface Geoscience
Lorentz Center@omega
April 20–24, 2026

[1]Delft University of Technology

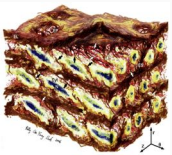
Artificial Intelligence in Science and Engineering

Medical Imaging

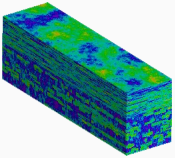
Epidemiology



Biomedical Engineering

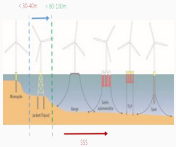


Geoscience



Fluid Mechanics

Civil Engineering



Unsupervised Learning
Use weak inductive biases to uncover structure from data

Inverse Problems
Use strong inductive biases to infer variables from data

Inference
Predict complex, nonlinear relations from data

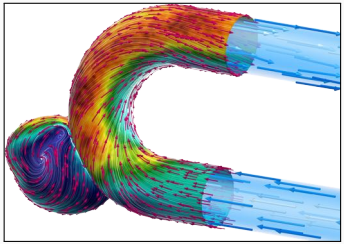
Challenges

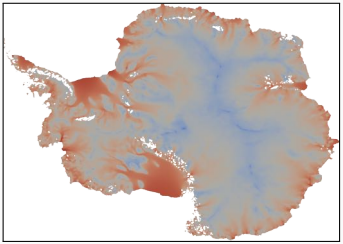
- Data are often **scarce** and **noisy**, yet we must deliver **accuracy**, **reliability**, and **robustness**
- **Multi-physics**, **multi-scale** systems require robust coupling, **scalability**, and **scale adaptation** of learning algorithms

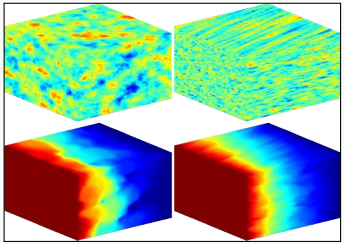
Opportunities

- Integrating **physics** with **data-driven models** enhances **generalization**, **interpretability**, and **reliability**
- Encoding **physical laws** enables **trustworthy predictions** and efficient surrogates for **real-time predictions and control** (digital twins)

Scientific Computing and Machine Learning







Numerical methods

Based on physical models

+ Robust and generalizable
− Require availability of mathematical models

Machine learning models

Driven by data

+ Do not require mathematical models
− Sensitive to data, limited extrapolation capabilities

Scientific machine learning

Combining the strengths and **compensating the weaknesses** of the individual approaches:

numerical methods	**improve**	machine learning techniques
machine learning techniques	**assist**	numerical methods

Scientific Machine Learning as a Standalone Field

N. Baker, A. Frank, T. Bremer, A. Hagberg, Y. Kevrekidis, H. Najm, M. Parashar, A. Patra, J. Sethian, S. Wild, K. Willcox, and S. Lee. **Workshop Report on Basic Research Needs for Scientific Machine Learning: Core Technologies for Artificial Intelligence.** USDOE Office of Science (SC), Washington, DC (United States), 2019.

Priority Research Directions

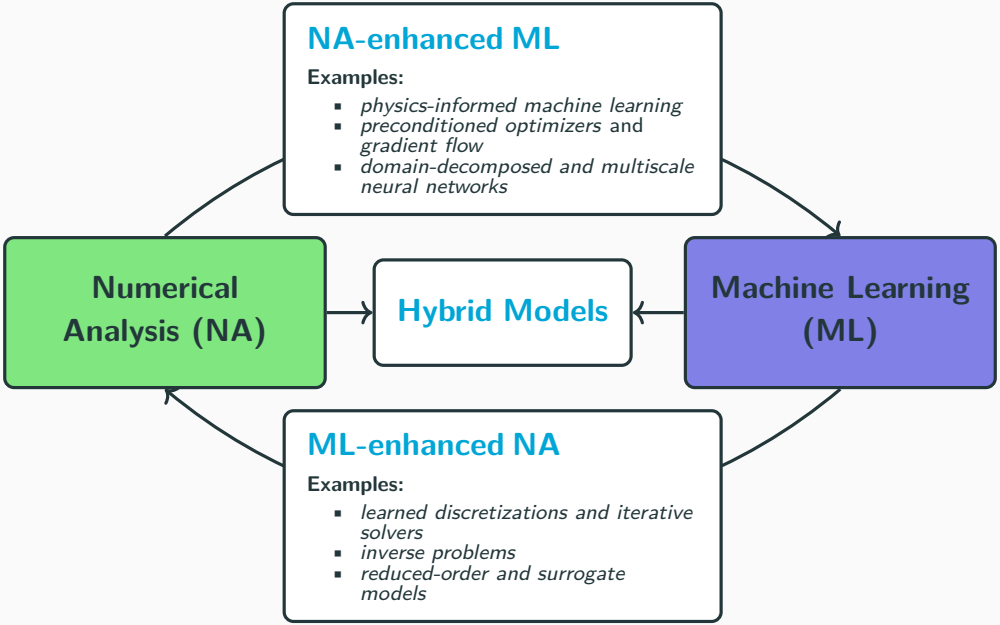
Foundational research themes:

- Domain-awareness
- Interpretability
- Robustness

Capability research themes:

- Massive scientific data analysis
- Machine learning-enhanced modeling and simulation
- Intelligent automation and decision-support for complex systems

Scientific Machine Learning Techniques



Outline

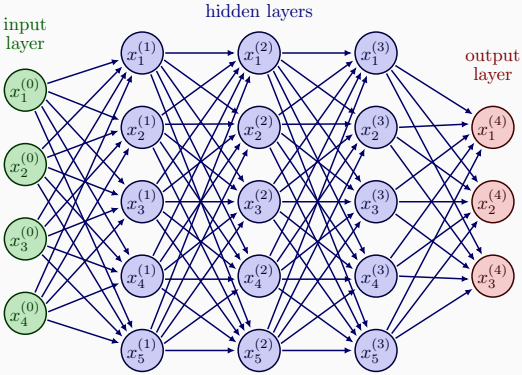
1 Neural Networks

2 Physics-Informed Neural Networks

3 Surrogate Models & Operator Learning

Neural Networks

Feedforward Neural Networks



Each layer applies an **affine map** followed by a **nonlinearity**:

$$x^{(j+1)} = \sigma\big(W^{(j+1)}x^{(j)} + b^{(j+1)}\big)$$

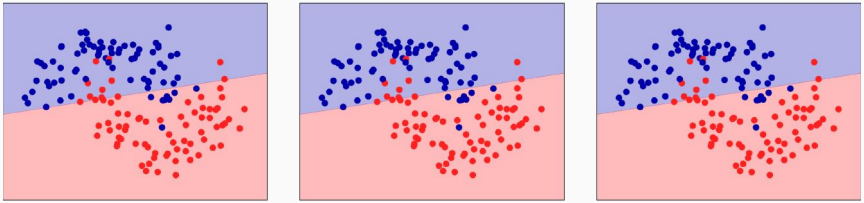
Here, j denotes the **layer index**.

Key ingredients:

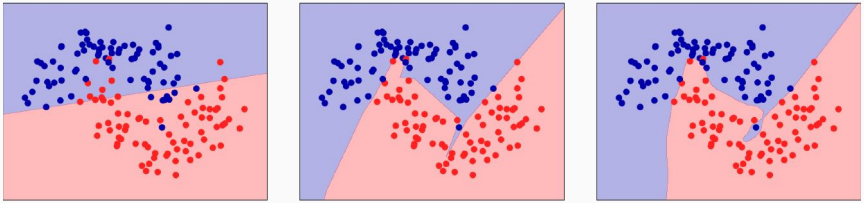
- **Trainable parameters** $\boldsymbol{\theta}$: weights $W^{(j+1)}$ and biases $b^{(j+1)}$
- **Activation function** σ: introduces the **nonlinearity**

Why Nonlinearity Matters

no σ



$\sigma(x) = \max\{0, x\}$

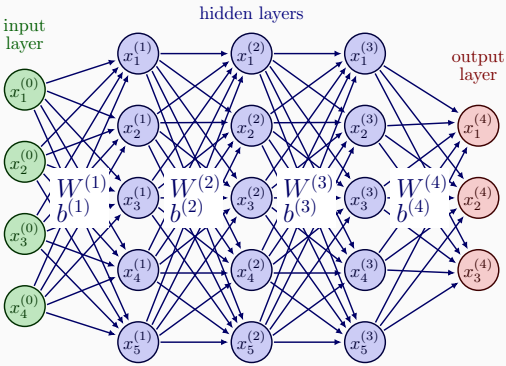


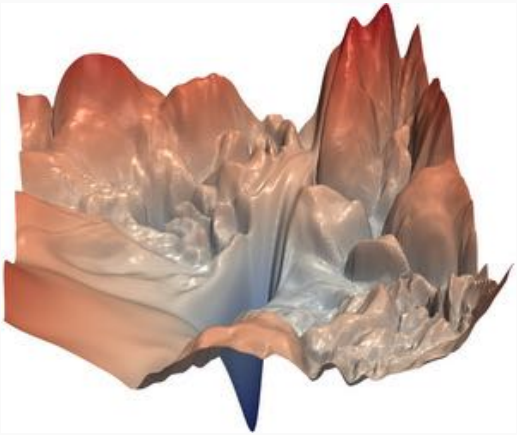
1 neuron | 10 neurons | 100 neurons

Key Point

Without the nonlinearity σ, stacked layers reduce to a single affine map.

Training a Neural Network



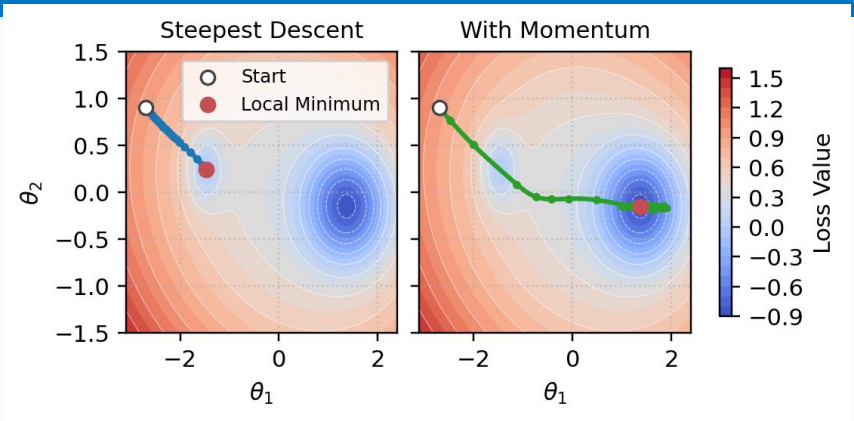


Minimize

$$\sum_{i=1}^{n} (y_i - \mathrm{ANN}(x_i))^2$$

with respect to **weights** $W^{(j)}$ and **biases** $b^{(j)}$ of the neural network ANN.

Training Using Gradient-Based Optimization



Training is typically performed using **gradient-based methods**:

$$\theta^{(k+1)} = \theta^{(k)} - \eta \nabla_\theta \mathcal{L}(\theta^{(k)})$$

where $\eta > 0$ is the **learning rate**.

Physics-Informed Neural Networks

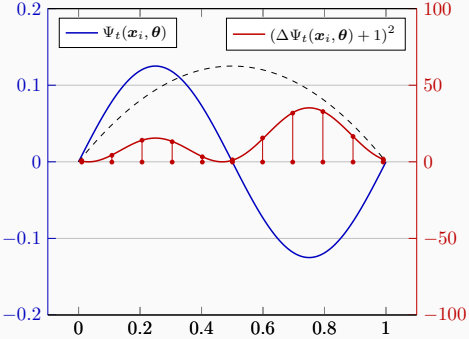
Physics-Informed Neural Networks (PINNs) – Idea

In **Lagaris et al. (1998)**, the authors solve the **boundary value problem**

$$-\Delta\Psi_t(\boldsymbol{x},\boldsymbol{\theta}) = 1 \text{ on } [0,1],$$
$$\Psi_t(0,\boldsymbol{\theta}) = \Psi_t(1,\boldsymbol{\theta}) = 0,$$

via a **collocation approach**:

$$\min_{\boldsymbol{\theta}} \sum_{\boldsymbol{x}_i} (\Delta\Psi_t(\boldsymbol{x}_i,\boldsymbol{\theta}) + 1)^2$$



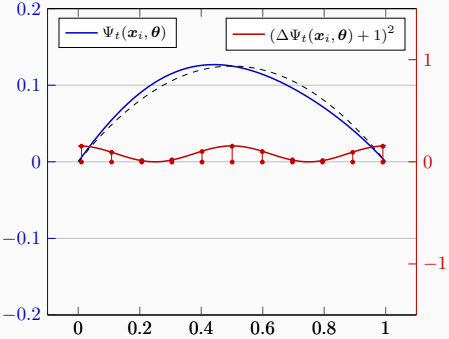
$(\Delta\Psi_t(\boldsymbol{x}_i,\boldsymbol{\theta}) + 1)^2 >> 0$

Boundary conditions ...

... can be **enforced explicitly** via the ansatz:

$$\Psi_t(\boldsymbol{x},\boldsymbol{\theta}) = A(\boldsymbol{x}) + F(\boldsymbol{x}, \mathrm{NN}(\boldsymbol{x},\boldsymbol{\theta}))$$

- A **satisfies the boundary conditions**
- F **does not contribute to the boundary conditions**



$(\Delta\Psi_t(\boldsymbol{x}_i,\boldsymbol{\theta}) + 1)^2 \approx 0$

Physics-Informed Neural Networks (PINNs)

In the **physics-informed neural network (PINN)** approach introduced by **Raissi et al. (2019)**, a **neural network** is employed to **discretize a partial differential equation**

$$\mathcal{N}[u] = f, \quad \text{in } \Omega.$$

PINNs use a **hybrid loss function**:

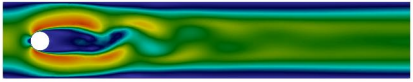
$$\mathcal{L}(\boldsymbol{\theta}) = \omega_{\text{data}}\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) + \omega_{\text{PDE}}\mathcal{L}_{\text{PDE}}(\boldsymbol{\theta}),$$

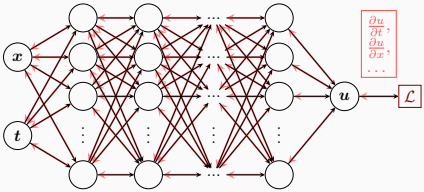
where ω_{data} and ω_{PDE} are **weights** and

$$\mathcal{L}_{\text{data}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{data}}} \sum_{i=1}^{N_{\text{data}}} \left(u(\hat{\boldsymbol{x}}_i, \boldsymbol{\theta}) - u_i\right)^2,$$

$$\mathcal{L}_{\text{PDE}}(\boldsymbol{\theta}) = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} \left(\mathcal{N}[u](\boldsymbol{x}_i, \boldsymbol{\theta}) - f(\boldsymbol{x}_i)\right)^2.$$

See also **Dissanayake and Phan-Thien (1994); Lagaris et al. (1998)**.





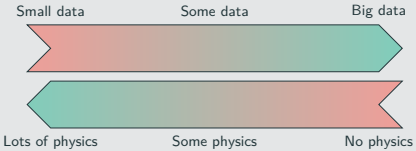
Advantages

- "Meshfree"
- Small data
- Generalization properties
- High-dimensional problems
- Inverse and parameterized problems

Drawbacks

- Training cost and robustness
- Convergence is not well understood
- Difficulties with scalability and multiscale problems

Hybrid loss



- **Known solution values** can be included in $\mathcal{L}_{\text{data}}$
- **Initial and boundary conditions** are also included in $\mathcal{L}_{\text{data}}$

Main Drivers of the Total Error of PINNs

Estimate of the total error (Mishra and Molinaro (2022))

The total error (or generalization error) satisfies

$$\mathcal{E}_G \leq C_{\text{PDE}} \mathcal{E}_T + C_{\text{PDE}} C_{\text{quad}}^{1/p} N^{-\alpha/p}$$

- $\mathcal{E}_G = \mathcal{E}_G(\boldsymbol{X}, \boldsymbol{\theta}) := \|\mathbf{u} - \mathbf{u}^*\|_V$ **general. error** (V Sobolev space, $\boldsymbol{X}$ training data set)
- $\mathcal{E}_T$ **training error** (l^p **loss** of the residual of the PDE)
- N **number of the training points** and α **convergence rate of the quadrature**
- C_{PDE} and C_{quad} **constants** depending on the **PDE**, **quadrature**, and **neural network**

- **Training error**: optimization is **challenging** due to the **highly nonconvex** loss landscape of PINNs.
- **Sampling error**: unlike classical numerical methods, standard quadrature rules **poorly approximate** neural-network trial functions.

Adaptive Sampling – Motivation

The **number and distribution of the collocation points in the PDE loss $\mathcal{L}_{\text{PDE}}$ have a significant influence on the accuracy of the PINN solution**. Since the **computational work grows with the number of collocation points**, the **effective placement of the collocation points** is important.

Burgers' equation in 1D ($\nu = 0.01/\pi$)

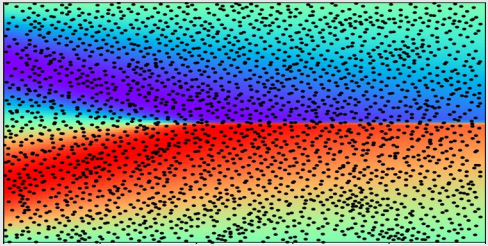
Consider Burgers' equation in one spatial dimension:

$$\frac{\partial u}{\partial t} + u\frac{\partial u}{\partial x} = \nu\frac{\partial^2 u}{\partial x^2}, \qquad x \in [-1, 1], \quad t \in [0, 1]$$

$$u(x, 0) = -\sin(\pi x) \quad u(-1, t) = u(1, t) = 0.$$

sampling method	L_2 relative error		mean
2500 coll. points	mean	1 SD	runtime [s]
uniform grid	25.9%	14.2%	425
Hammersley grid	0.61%	0.53%	443
random resampling	0.40%	0.35%	**423**
residual-based	**0.11%**	**0.05%**	450

Implementation based on DeepXDE with the PyTorch (v1.12.1) backend; cf. **Visser, Heinlein, and Giovanardi (2026)**



Similar results were reported in an earlier study: **Wu et al. (2023)**

Overview of Various Sampling Schemes (Not Exhaustive)

Non-adaptive sampling

- **Equispaced uniform grid**
- **Uniformly random sampling**, using a pseudo-random number generator (e.g., PCG-64 **O'Neill (2014)**)
- **Latin hypercube sampling** (**McKay, Beckman, and Conover (2000); Stein (1987)**)

Quasi-random low-discrepancy sequences:

- **Halton sequence** (**Halton (1960)**)
- **Hammersley sequence** (**Hammersley (1964)**)
- **Sobol sequence** (**Sobol' (1967)**)

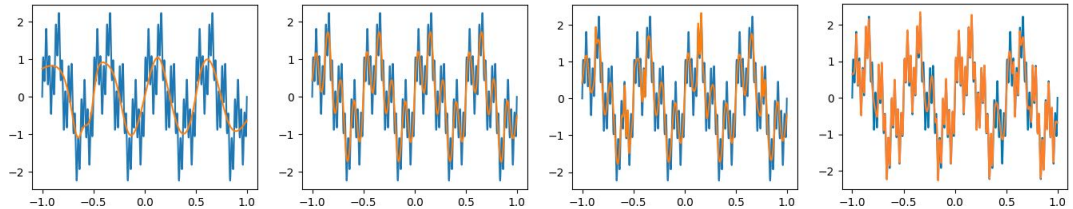
Adaptive sampling

- **Residual-based Adaptive Refinement (RAR)** (**Lu et al. (2021)**): placement of **additional points** in regions with the largest PDE residuals
- **Probability Density Function (PDF)** (**Nabian, Gladstone, and Meidani (2021)**): **randomly resample all points** based on a **PDF proportional to the residual** (linear)
- **Residual-based Adaptive Distribution (RAD)** (**Wu et al. (2023)**): **all collocation points are resampled** using a **PDF based on the residual** (nonlinear)
- **Residual-based Adaptive Refinement with Distribution (RAR-D)** (**Wu et al. (2023)**): **sampling of additional collocation points** using the **PDF used in RAD**

(RAD and RAR-D are based on, or extend, the PDF approach.)

Spectral Bias of Neural Networks

Rahaman et al. (2019) observed the **spectral bias of neural networks**: during training, they **learn low-frequency functions faster than high-frequency functions**; see also **Cao et al. (2021), Wang et al. (2022), Hong et al. (arXiv 2022), Xu et al. (2024), . . .**



For a mean-squared loss $\mathcal{C}(\boldsymbol{\theta}) = \frac{1}{2N}\|\boldsymbol{r}(\boldsymbol{\theta})\|^2$ with error/PDE residual $\boldsymbol{r}(\boldsymbol{\theta})$ and Jacobian $\boldsymbol{J} = \frac{\partial \boldsymbol{r}(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$, gradient descent yields

$$\boldsymbol{\theta}_{k+1} = \boldsymbol{\theta}_k - \eta \nabla_{\boldsymbol{\theta}} \mathcal{C}(\boldsymbol{\theta}_k) = \boldsymbol{\theta}_k - {}^{\eta}/_{N}\, \boldsymbol{J}_k^\top \boldsymbol{r}(\boldsymbol{\theta}_k)$$

Linearization gives

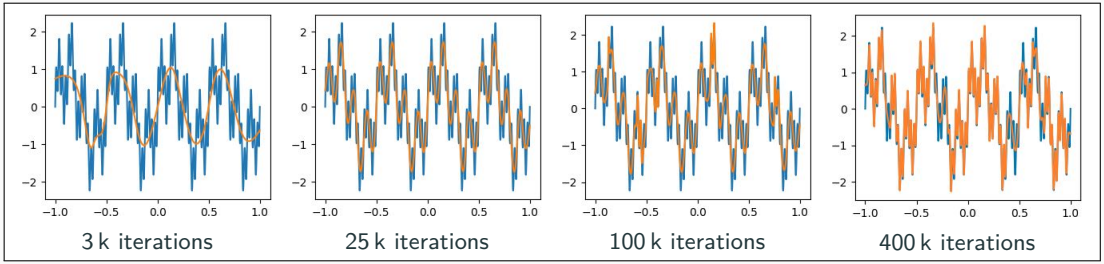
$$\boldsymbol{r}(\boldsymbol{\theta}_{k+1}) \approx \boldsymbol{r}(\boldsymbol{\theta}_k) + \boldsymbol{J}_k(\boldsymbol{\theta}_{k+1} - \boldsymbol{\theta}_k) = \boldsymbol{r}(\boldsymbol{\theta}_k) - {}^{\eta}/_{N} \boldsymbol{J}_k \boldsymbol{J}_k^\top \boldsymbol{r}(\boldsymbol{\theta}_k).$$

Here, $\boldsymbol{\Theta}_k := {}^{1}/_{N} \boldsymbol{J}_k \boldsymbol{J}_k^\top$ is the (finite-width) **neural tangent kernel (NTK)**. For **infinitely wide neural networks** or **infinitesimal step sizes**, we obtain a **constant** NTK, which yields the residual propagation:

$$\boldsymbol{r}_k \approx (\boldsymbol{I} - \eta\boldsymbol{\Theta})^k \boldsymbol{r}_0 = \boldsymbol{V}^\top (\boldsymbol{I} - \eta\boldsymbol{\Lambda})^k \boldsymbol{V}\boldsymbol{r}_0.$$

Spectral Bias of Neural Networks

Rahaman et al. (2019) observed the **spectral bias of neural networks**: during training, they **learn low-frequency functions faster than high-frequency functions**; see also **Cao et al. (2021), Wang et al. (2022), Hong et al. (arXiv 2022), Xu et al. (2024), . . .**



Scaling of PINNs for a Simple ODE Problem

Solve

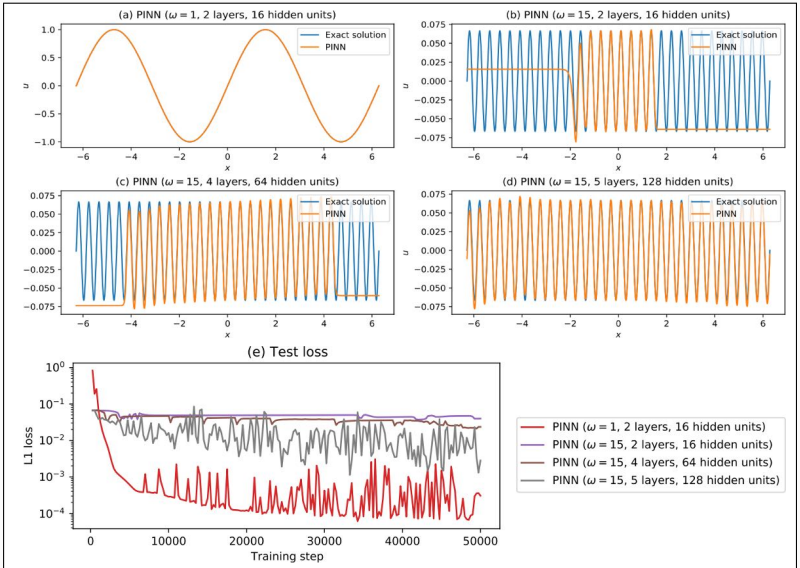
$$\begin{aligned} u' &= \cos(\omega x), \\ u(0) &= 0, \end{aligned}$$

for different values of ω using **PINNs with varying network capacities**.

Scaling issues

- Large computational domains
- Small frequencies

Cf. **Moseley, Markham, and Nissen-Meyer (2023)**



(a) 321 free parameters (d) 66 433 free parameters

Surrogate Models & Operator Learning

Surrogate Modeling — Operator Learning

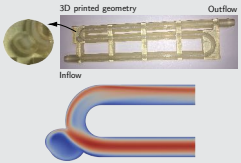
Surrogate models

Surrogate models replace **costly high-fidelity simulations** by cheap approximations for **fast repeated queries**.

Operator learning

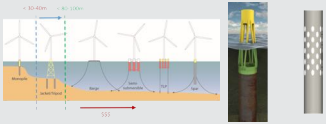
Learn an operator $\mathcal{G} : \mathcal{X} \to \mathcal{Y}$ between **function spaces** $\mathcal{X}$ and $\mathcal{Y}$, viewed as **Banach spaces**.

Aneurysm hemodynamics



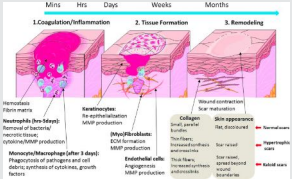
Patient-specific artery → wall shear stresses

Perforated monopiles



Monopile geometry → wave loading

Post-burn wound healing



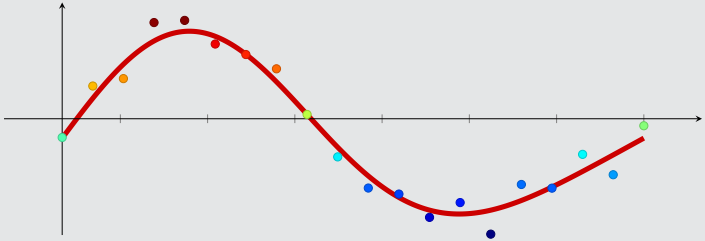
Wound parameters → wound evolution

Summary

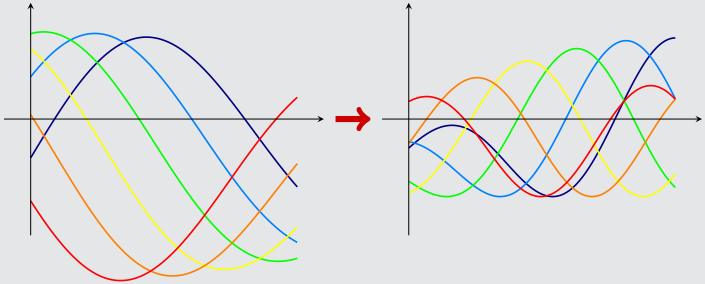
In all three cases, the input parametrizes the configuration and the output is the solution of an **(initial-)boundary value problem**. Operator learning learns this map between function spaces.

Function Versus Operator Learning

Function learning

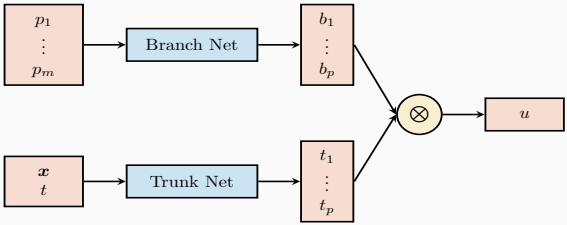


Operator learning



Deep Operator Networks (DeepONets / DONs)

Neural operators learn operators between function spaces using neural networks. Here, we learn the **solution operator** of an initial-boundary value problem parametrized with $p_1, \dots, p_m$ using **DeepONets** as introduced in **Lu et al. (2021)**.



Single-layer case

The DeepONet architecture is based on the **single-layer case** analyzed in **Chen and Chen (1995)**. In particular, the authors show **universal approximation properties for continuous operators**.

The architecture is based on the following ansatz for representing the parametrized solution

$$u_{(p_1,\dots,p_m)}(\boldsymbol{x}, t) = \sum_{i=1}^{p} \underbrace{b_i(p_1, \dots, p_m)}_{\text{branch}} \cdot \underbrace{t_i(\boldsymbol{x}, t)}_{\text{trunk}}$$

Physics-informed DeepONets

DeepONets are **compatible with the PINN approach** but **physics-informed DeepONets (PI-DeepONets)** are challenging to train.

Other operator learning approaches

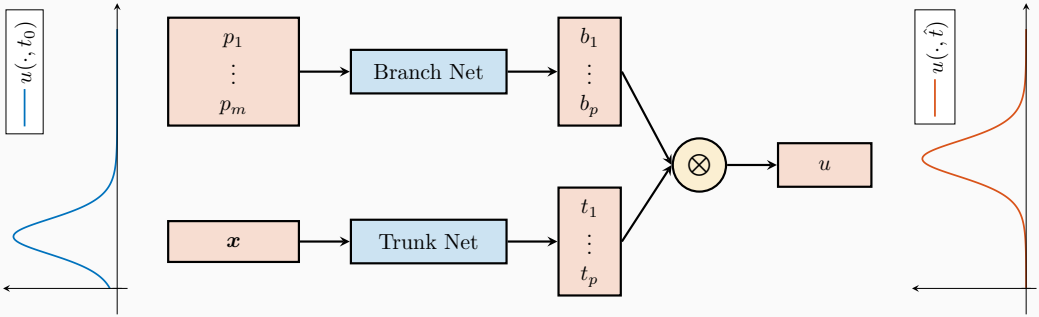
- **FNOs**: **Li et al. (2021)**
- **PCA-Net**: **Bhattacharya et al. (2021)**
- **Random features**: **Nelsen and Stuart (2021)**
- **CNOs**: **Raonić et al. (2023)**

How a DeepONet Maps Between Function Spaces

To illustrate how a DeepONet operates, we consider the **Korteweg–de Vries (KdV) equation**

$$\frac{\partial u}{\partial t} = -u\frac{\partial u}{\partial x} - 0.01\frac{\partial^3 u}{\partial x^3},$$

which models unidirectional waves in shallow water. Our goal is to train a DeepONet that predicts the wave profile at a future time $\hat{t}$ from the observed height profile $u(\cdot, t_0)$.



Here, the forecast time $\hat{t}$ is fixed to keep the learning task simple. A **more general neural operator** can take the target time as an additional input to the trunk network.

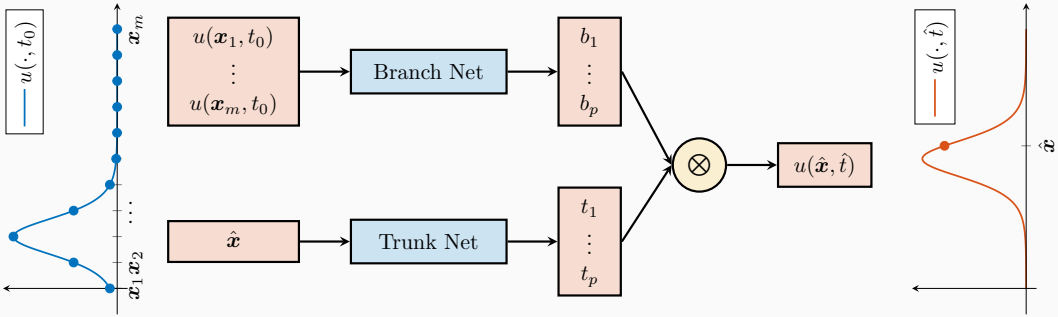
Cf. **Heinlein and Taraz (subm. 2026; arXiv:2602.21910)**

How a DeepONet Maps Between Function Spaces

To illustrate how a DeepONet operates, we consider the **Korteweg–de Vries (KdV) equation**

$$\frac{\partial u}{\partial t} = -u\frac{\partial u}{\partial x} - 0.01\frac{\partial^3 u}{\partial x^3},$$

which models unidirectional waves in shallow water. Our goal is to train a DeepONet that predicts the wave profile at a future time $\hat{t}$ from the observed height profile $u(\cdot, t_0)$.

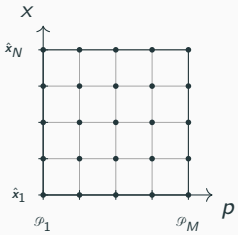


Here, the forecast time $\hat{t}$ is fixed to keep the learning task simple. A **more general neural operator** can take the target time as an additional input to the trunk network.

Cf. **Heinlein and Taraz (subm. 2026; arXiv:2602.21910)**

DeepONet – Error Decomposition

Given **fixed parametrizations** $\mathscr{P}_1, \dots, \mathscr{P}_M$ and **evaluation points** $\hat{\boldsymbol{x}}_1, \dots, \hat{\boldsymbol{x}}_N$, the reference targets form the matrix



$$\boldsymbol{A} = \begin{bmatrix} u_{\mathscr{P}_1}(\hat{\boldsymbol{x}}_1) & \cdots & u_{\mathscr{P}_M}(\hat{\boldsymbol{x}}_1) \\ \vdots & \ddots & \vdots \\ u_{\mathscr{P}_1}(\hat{\boldsymbol{x}}_N) & \cdots & u_{\mathscr{P}_M}(\hat{\boldsymbol{x}}_N) \end{bmatrix}$$

and the outputs of the DeepONet can be written as

$$\tilde{\boldsymbol{A}} = \underbrace{\begin{bmatrix} t_1(\hat{\boldsymbol{x}}_1) & \cdots & t_m(\hat{\boldsymbol{x}}_1) \\ \vdots & \ddots & \vdots \\ t_1(\hat{\boldsymbol{x}}_N) & \cdots & t_m(\hat{\boldsymbol{x}}_N) \end{bmatrix}}_{=:\boldsymbol{T}} \underbrace{\begin{bmatrix} b_1(\mathscr{P}_1) & \cdots & b_1(\mathscr{P}_M) \\ \vdots & \ddots & \vdots \\ b_m(\mathscr{P}_1) & \cdots & b_m(\mathscr{P}_M) \end{bmatrix}}_{=:\boldsymbol{B}^\top} = \begin{bmatrix} \boldsymbol{t}_1 & \cdots & \boldsymbol{t}_m \end{bmatrix} \begin{bmatrix} \boldsymbol{b}_1^\top \\ \vdots \\ \boldsymbol{b}_m^\top \end{bmatrix},$$

where $\boldsymbol{B}$ and $\boldsymbol{T}$ are discretizations of the branch and trunk net outputs. Similar to **Lanthaler et al. (2022)**, we derive an **error decomposition into branch and trunk errors**

$$\mathscr{E} := \|\boldsymbol{A} - \tilde{\boldsymbol{A}}\|_F^2 = \underbrace{\|\boldsymbol{T}(\boldsymbol{T}^+\boldsymbol{A} - \boldsymbol{B}^\top)\|_F^2}_{=:\mathscr{E}_B} + \underbrace{\|(\boldsymbol{I} - \boldsymbol{T}\boldsymbol{T}^+)\boldsymbol{A}\|_F^2}_{=:\mathscr{E}_T},$$

where $\|\cdot\|_F$ is the **Frobenius norm** and $\boldsymbol{T}^+$ is the **pseudoinverse** of $\boldsymbol{T}$.

Analyzing the DeepONet Using the Singular Value Decomposition

A **singular value decomposition (SVD)** of the target matrix $\boldsymbol{A}$ (rank k) reads as follows:

$$\boldsymbol{A} = \boldsymbol{\Phi}\boldsymbol{\Sigma}\boldsymbol{V}^\top = \begin{bmatrix} \phi_1 & \cdots & \phi_k \end{bmatrix} \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_k \end{bmatrix} \begin{bmatrix} \boldsymbol{v}_1^\top \\ \vdots \\ \boldsymbol{v}_k^\top \end{bmatrix} = \sum_{j=1}^{k} \sigma_j \phi_j \boldsymbol{v}_j^\top,$$

where $\boldsymbol{\Phi}$ and $\boldsymbol{V}$ are semi-orthogonal matrices. The **Eckart–Young theorem** states that the **best low-rank approximation of a matrix in the Frobenius norm is given by truncating its SVD**.

The structure is the same for the discretized DeepONet

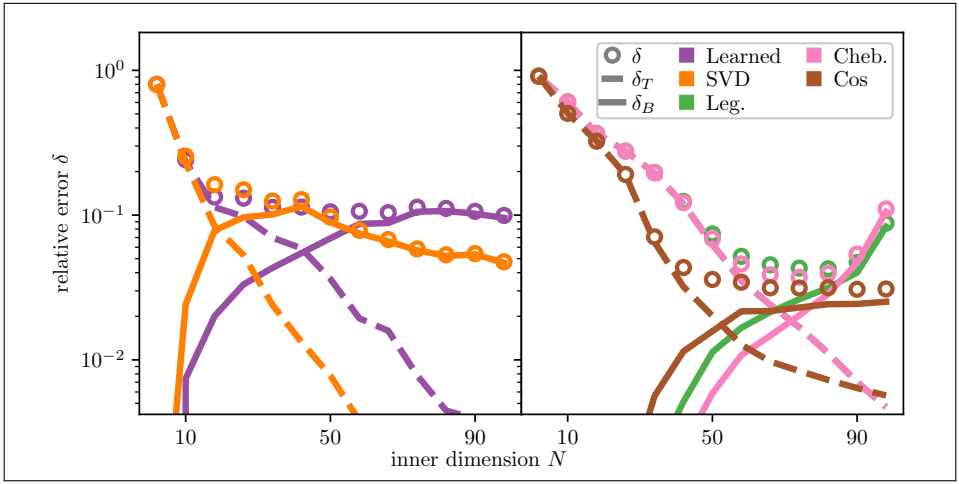
$$\tilde{\boldsymbol{A}} = \begin{bmatrix} \boldsymbol{t}_1 & \cdots & \boldsymbol{t}_m \end{bmatrix} \begin{bmatrix} \boldsymbol{b}_1^\top \\ \vdots \\ \boldsymbol{b}_m^\top \end{bmatrix}.$$

Therefore, we can minimize the discrete approximation error on the training data by choosing:

$$\boldsymbol{t}_i = \phi_i \quad \text{and} \quad \boldsymbol{b}_i = \sigma_i \boldsymbol{v}_i.$$

DeepONet – Error Decomposition Results for the KdV Equation

Results for the **KdV equation** with $t_0 = 0.0$ and $\hat{t} = 0.2$. 900 training and 100 test configurations.



total error δ **trunk error** δ_T **branch error** δ_B

Cf. **Heinlein and Taraz (subm. 2026; arXiv:2602.21910)**

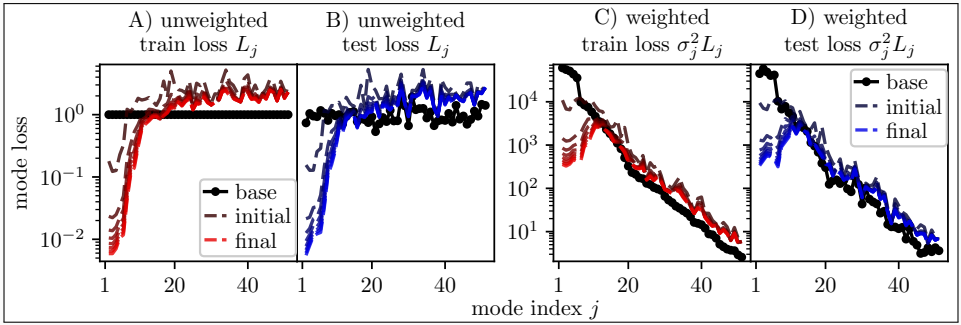
DeepONet – Branch Error

Weighted and unweighted mode losses

Using the left singular vectors as the trunk basis, the **branch error** becomes

$$\mathcal{E}_B = \sum_{i=1}^{m} \sigma_i^2 \underbrace{\|b_i - v_i\|_2^2}_{=:L_i}.$$

We call L_i the **unweighted mode loss**. The quantity $\sigma_i^2 L_i$ is the **(weighted) mode loss**, i.e., the contribution of the ith mode to the branch error $\mathcal{E}_B$. This choice of **left singular vectors as the trunk basis** is denoted **POD-DeepONet** in **Lu et al. (2022)**.

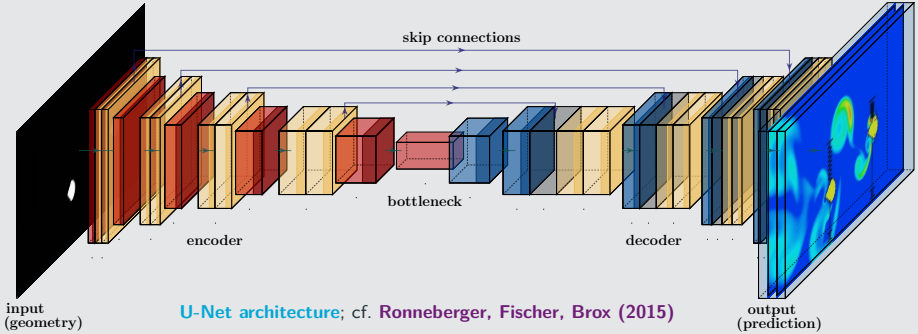


Cf. **Heinlein and Taraz (subm. 2026; arXiv:2602.21910)**

Convolutional Neural Network-Based Surrogate Model

CNN-based approach

We employ a **convolutional neural network (CNN)** (**LeCun (1998)**) to predict the stationary flow field given an **image of the geometry as input**.



U-Net architecture; cf. Ronneberger, Fischer, Brox (2015)

Related works (non-exhaustive)

- Guo, Li, Iorio (2016)
- Niekamp, Niemann, Schröder (2022)
- Stender, Ohlsen, Geisler, Chabchoub, Hoffmann, Schlaefer (2022)

Operator learning (non-exhaustive)

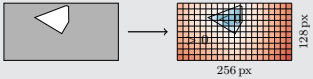
- FNOs: Li et al. (2021)
- PCA-Net: Bhattacharya et al. (2021)
- Random features: Nelsen and Stuart (2021)
- CNOs: Raonić et al. (2023)

Comparison of OpenFOAM® and CNN Predictions (Relative Error 2 %)

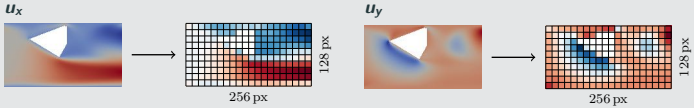
We automatically generate geometries and compute the corresponding flow fields using OpenFOAM®.

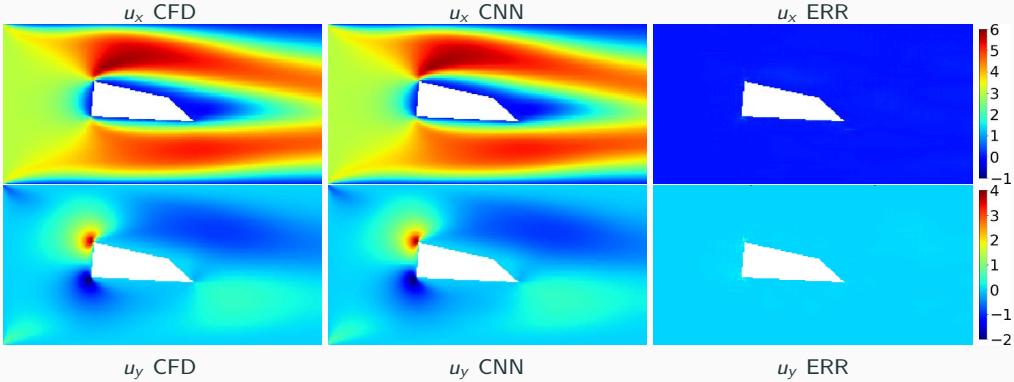
Input data

SDF (Signed Distance Function)



Output data





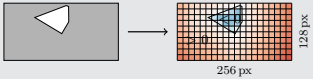
Cf. **Eichinger, Heinlein, Klawonn (2021, 2022)**.

Comparison of OpenFOAM® and CNN Predictions (Relative Error 14 %)

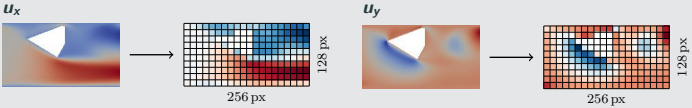
We automatically generate geometries and compute the corresponding flow fields using OpenFOAM®.

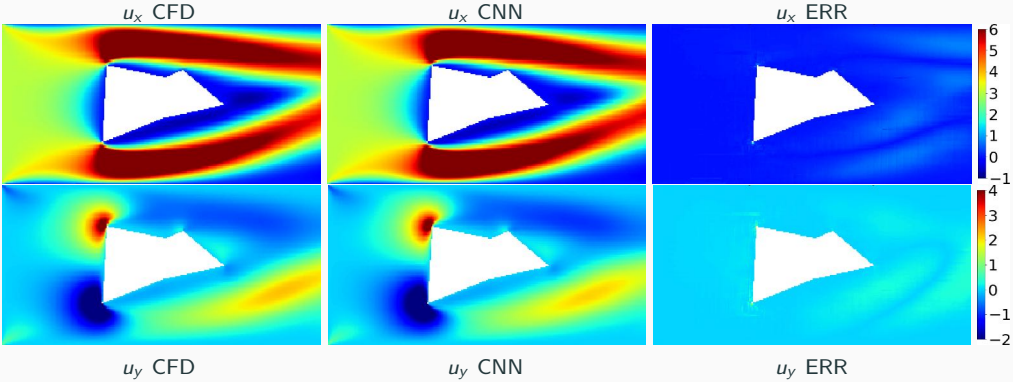
Input data

SDF (Signed Distance Function)



Output data





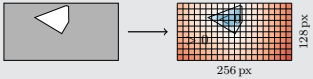
Cf. **Eichinger, Heinlein, Klawonn (2021, 2022)**.

Comparison of OpenFOAM® and CNN Predictions (Relative Error 31 %)

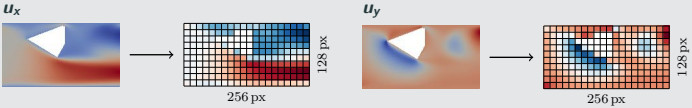
We automatically generate geometries and compute the corresponding flow fields using OpenFOAM®.

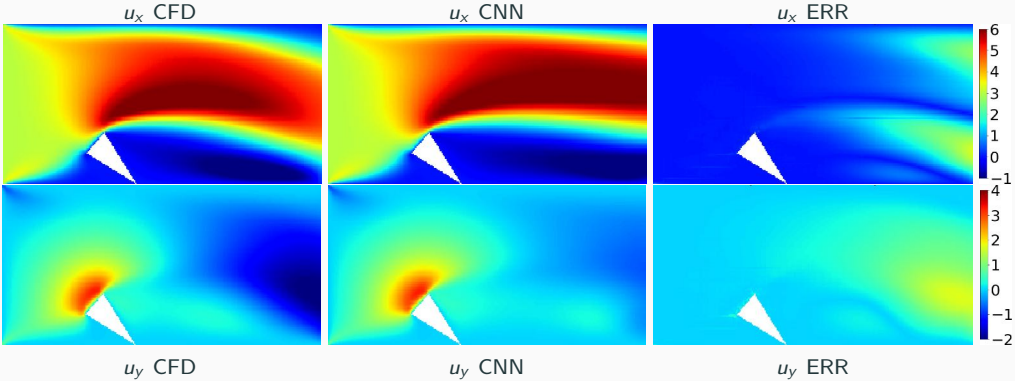
Input data

SDF (Signed Distance Function)



Output data





Cf. **Eichinger, Heinlein, Klawonn (2021, 2022)**.

Unsupervised Learning Approach – PDE Loss Using Finite Differences

Physics-informed loss function

We train the CNN by incorporating the **PDE residuals**, discretized via finite differences, into the **loss function**:

$$\mathcal{L}_{\text{PDE}} = \frac{1}{N_{\text{PDE}}} \sum_{i=1}^{N_{\text{PDE}}} \|\mathcal{R}(u_{\text{CNN}}, p_{\text{CNN}})\|_2^2$$

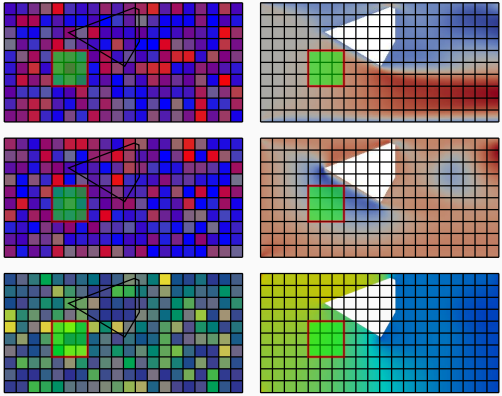
Here, N_{PDE} is the number of training configs.

Cf. **Raissi et al. (2019), Dissanayake and Phan-Thien (1994), Lagaris et al. (1998)**.

We explcitly enforce boundary conditions on the output image → **hard constraints**

error	$\frac{\|u_{NN}-u\|_2}{\|u\|_2}$	$\frac{\|p_{NN}-p\|_2}{\|p\|_2}$	mean residual		# epochs trained
			moment.	mass	
train.	1.43 %	7.30 %	$1.0 \cdot 10^{-1}$	$1.5 \cdot 10^{0}$	500
val.	2.52 %	8.67 %	$1.2 \cdot 10^{-1}$	$1.5 \cdot 10^{0}$	
train.	5.03 %	11.63 %	$3.2 \cdot 10^{-2}$	$7.7 \cdot 10^{-2}$	**5 000**
val.	5.18 %	11.60 %	$4.2 \cdot 10^{-2}$	$1.1 \cdot 10^{-1}$	

→ Errors are **comparable to the data-based approach**, but **training requires more epochs**.



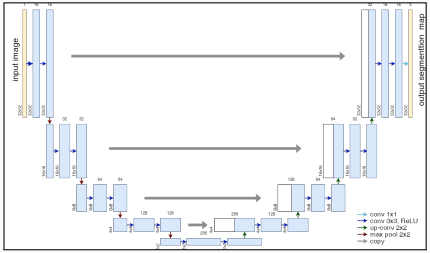
$\|\mathcal{R}(u_{\text{CNN}}, p_{\text{CNN}})\|^2 >> 0$ $\|\mathcal{R}(u_{\text{CNN}}, p_{\text{CNN}})\|^2 \approx 0$

Here, we consider ther **Navier–Stokes equations**:

$$\mathcal{R}(u_{\text{CNN}}, p_{\text{CNN}}) = \begin{bmatrix} -\nu \Delta \vec{u} + (u \cdot \nabla)\, \vec{u} + \nabla p \\ \nabla \cdot u \end{bmatrix}$$

Cf. **Grimm, Heinlein, Klawonn (2025)**.

Domain Decomposition-Based U-Net Architecture

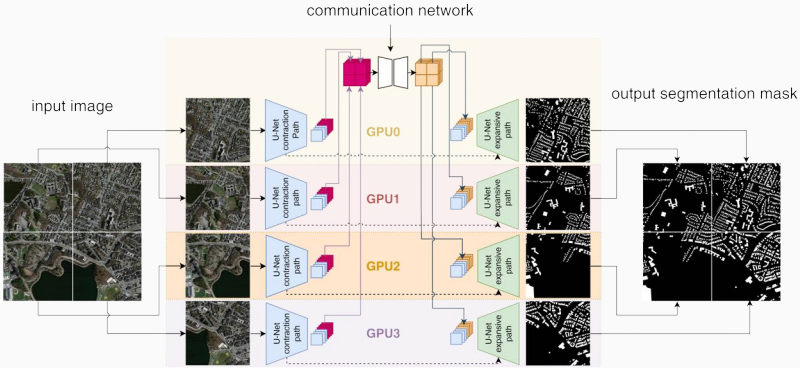


name	mem. feature maps		mem. weights	
	# of values	MB	# of values	MB
input block	268 M	1 024.0	38 848	**0.148**
encoder blocks	314 M	1 320	18 M	**72**
decoder blocks	754 M	3880	12 M	**47**
output block	3.1 M	12.0	195	**0.001**

Most U-Net memory is used by **feature maps**, **not weights**

→ **Decompose feature maps** to **distribute memory consumption**.

Cf. **Verburg, Heinlein, Cyr (2025).**



Thank you for your attention!

ELLIS Unit TU Delft

SRI Bridging NA and ML

Topical Activity Group SciML